Exercise 6.1: Central User Management with LDAP

Setup walkthrough

You will build a small NordTech network with one LDAP server and two desktops. You will manage users, groups, sudo rights and file ownership from one central place, see how least privilege applies to the directory itself, see what happens when the server fails, and finally secure the connection with TLS.

> **Before you start:** take a VM snapshot of every machine. Several steps edit authentication files, and a mistake can lock you out.

---

0. The plan

Machine	Role	Hostname
Ubuntu **Server**	LDAP server (slapd)	`ldap.nordtech.dk`
Ubuntu **Desktop 1**	LDAP client	`desktop1`
Ubuntu **Desktop 2**	LDAP client	`desktop2`

All three VMs must be on the **same network** and able to reach each other (for example, the same host-only or bridged network). Check with `ping`.

Values used in this guide. If you change them, change them everywhere.

Item	Value
DNS domain	`nordtech.dk`
Base DN	`dc=nordtech,dc=dk`
Admin DN	`cn=admin,dc=nordtech,dc=dk`
Server IP	`192.168.X.X` *(replace with your server's real IP)*
UIDs for LDAP users	start at **10000**
GIDs for LDAP groups	start at **5000**

The NordTech people.

Person	User name	UID	Primary group	Role in this exercise
Maria Hansen	`maria`	10001	`sales`	the new colleague, ordinary user
Jens Nielsen	`jens`	10002	`economy`	the contractor who will leave
Lars Madsen	`lars`	10003	`support`	administrator, also in `ldapadmins`
Sofie Larsen	`sofie`	10004	`support`	added later in Part 4

Why high numbers? Local Ubuntu users start at 1000. If an LDAP user had UID 1001 and a local user had it too, they would be treated as the same person by the file system. High numbers keep the two worlds apart.

Understanding the LDAP commands

You will use the same few commands and options again and again. Read this section once, then come back to it when a command looks confusing.

The tools

Command	What it does	Talks to the server over the network?
`ldapsearch`	Searches the directory and prints entries	Yes
`ldapadd`	Adds new entries from an LDIF file	Yes
`ldapmodify`	Changes existing entries (add, replace or delete attributes) from an LDIF file	Yes
`ldapdelete`	Deletes entries	Yes
`ldappasswd`	Sets or changes a user's password	Yes
`ldapwhoami`	Shows who the server thinks you are after logging in (a good password test)	Yes
`slapcat`	Dumps the whole database as LDIF by reading the server's files directly (needs `sudo`, runs on the server)	No

`ldapadd` is simply `ldapmodify -a` (add mode).

The options

Connecting and logging in

Option	Meaning
`-x`	Use **simple authentication** (a DN and a password). Without it the tools try SASL, which usually fails with errors like `ldap_sasl_interactive_bind: Can't contact LDAP server`.
`-H URI`	**Host**: the address of the server. `ldap://ldap.nordtech.dk` is the normal port 389. `ldaps://...` is LDAP inside TLS (port 636). `ldapi:///` is the local Unix socket on the server itself, used together with `-Y EXTERNAL` and `sudo`.
`-D "DN"`	The **bind DN**: the entry you log in as, for example `cn=admin,dc=nordtech,dc=dk` or `uid=maria,ou=people,dc=nordtech,dc=dk`.
`-W`	**Prompt** for the password of the `-D` user. The password is not shown and does not end up in your shell history.
`-w PASSWORD`	Give the password on the command line. Avoid it: it is visible in the process list and in your history.
`-Y EXTERNAL`	Use the SASL method **EXTERNAL**: the server trusts the operating system identity. With `sudo` and `-H ldapi:///` you are treated as root and may change the server's own configuration (`cn=config`), without any LDAP password.
`-Q`	**Quiet** SASL mode: no questions and no extra messages. Used with `-Y EXTERNAL`.
`-ZZ`	Use **START_TLS** (encrypt the connection) and stop with an error if that does not work. A single `-Z` tries TLS but carries on without it if it fails.

Searching (`ldapsearch`)

Option	Meaning
`-b "DN"`	The **base**: the entry where the search starts. `-b "dc=nordtech,dc=dk"` searches the whole tree. Without a base the server often answers *No such object*.
`-s scope`	How deep to search from the base: `base` (only the base entry), `one` (its direct children) or `sub` (everything below, the default).
`-L`, `-LL`, `-LLL`	**LDIF output** in three steps. `-L` prints LDIF. `-LL` also removes the comments. `-LLL` also removes the version line, so you get only clean entries.

Option	Meaning
`"(uid=maria)"`	The **filter**: which entries to find. The quotes keep the shell from reading the parentheses. Examples: `(uid=*)` all users, `(uidNumber=10001)` by number, `(&(objectClass=posixGroup)(cn=sales))` both conditions.
`userPassword` (a name after the filter)	**Attributes to show**. Without names you get all normal attributes. With names you only get those.

Changing things

Option	Meaning
`-f file.ldif`	Read the operations from an LDIF **file** instead of the keyboard.
`-S` (`ldappasswd`)	**Prompt** for the new password of the user whose DN you give at the end. Note: `-W` is for *your* (admin) password, `-S` is for the *new* one.
`-s PASSWORD` (`ldappasswd`)	Give the new password on the command line. Avoid it, as with `-w`.

`slapcat`

Option	Meaning
`-b "suffix"`	Which database to dump, identified by its base DN, for example `dc=nordtech,dc=dk`. Without it, `slapcat` dumps all databases.
`-l file`	Write the output to a **file** instead of the screen.

Environment variable

- `LDAPTLS_CACERT=/path/ca.crt` in front of a command tells the LDAP tools which CA certificate to trust for that one command.

Reading a whole command

```
ldapsearch -x -LLL -H ldap://ldap.nordtech.dk -b "dc=nordtech,dc=dk" \
  "(uid=maria)" uidNumber
```

Part	Meaning
`ldapsearch`	Search the directory
`-x`	Simple authentication (here anonymous, since there is no `-D`)
`-LLL`	Print clean LDIF without comments or version line
`-H ldap://ldap.nordtech.dk`	Connect to this server on port 389
`-b "dc=nordtech,dc=dk"`	Start in the root of the tree
`"(uid=maria)"`	Find the entry whose `uid` is `maria`
`uidNumber`	Show only this attribute (plus the DN)

The LDIF files

LDIF is the plain-text format used for `ldapadd` and `ldapmodify`. Entries are separated by a blank line:

Line	Meaning
`dn: ...`	The entry the operation is about. It is always the first line.
`changetype: modify`	Used in `ldapmodify` files: change an existing entry.

Line	Meaning
`add:` / `replace:` / `delete: attribute`	What to do with that attribute. `replace` sets the value, and creates the attribute if it is missing.
a line with only `-`	Separates several changes in the same entry.
`objectClass: ...`	The "type" of the entry. It decides which attributes are required and allowed, for example `posixAccount` for Linux users.

Part 1: The LDAP server

All commands in Part 1 run on the **Ubuntu Server**.

1.1 Name the server

```
sudo hostnamectl set-hostname ldap.nordtech.dk
```

Edit `/etc/hosts` and add a line with the server's own IP:

```
192.168.X.X   ldap.nordtech.dk   ldap
```

1.2 Install and configure slapd

```
sudo apt update
sudo apt install slapd ldap-utils
```

You will be asked for an administrator password. Choose one and **write it down**. Then run the configuration wizard:

```
sudo dpkg-reconfigure slapd
```

Answer the questions like this:

Question	Answer
Omit OpenLDAP server configuration?	**No**
DNS domain name	`nordtech.dk`
Organization name	`nordtech`
Administrator password	the one you wrote down
Remove the database when slapd is purged?	**No**
Move old database?	**Yes**

1.3 Check that the server works

```
sudo slapcat
```

You should see the top entry `dn: dc=nordtech,dc=dk`.

Check that the schemas you need are loaded (you should see `nis` and `inetorgperson` in the list):

```
sudo ldapsearch -Q -LLL -Y EXTERNAL -H ldapi:/// -b cn=schema,cn=config dn
```

The `nis` schema is what provides `posixAccount` and `posixGroup`, which Linux needs to treat an LDAP entry as a user or group.

1.4 Create the two containers (OUs)

(All the options in the commands below are explained in *Understanding the LDAP commands* above.)

Whichever way you create users later, the directory first needs two empty containers: `people` and `groups`. Create **`base.ldif`**:

```
dn: ou=people,dc=nordtech,dc=dk
objectClass: organizationalUnit
ou: people

dn: ou=groups,dc=nordtech,dc=dk
objectClass: organizationalUnit
ou: groups
```

```
ldapadd -x -D "cn=admin,dc=nordtech,dc=dk" -W -f base.ldif
```

1.5 Install the LDAP Account Manager (web interface)

LDAP Account Manager (LAM) lets you create and edit groups and users in a browser, so you do not have to write LDIF files by hand. It runs on top of Apache and PHP.

```
sudo apt install ldap-account-manager
```

This normally pulls in Apache and PHP as well. Check that the web server is running:

```
systemctl status apache2
```

Open a browser (on your own computer, or on a desktop VM) and go to:

```
http://192.168.X.X/lam
```

> **If the page is forbidden when you open it from another machine:** the LAM package can restrict access to certain networks in its Apache configuration. Look in `/etc/ldap-account-manager/apache.conf` (and `/etc/apache2/conf-enabled/`) for `Require ip` lines and add your VM network. Then run `sudo systemctl reload apache2`.

Configure LAM (one time)

1. On the login page, click **LAM configuration** (top right), then **Edit server profiles**.
2. Log in to the profile editor with the default profile password: **`lam`**.
3. **General settings tab:** under *Server settings*, the server address should be `ldap://localhost:389`. Set the **tree suffix** to `dc=nordtech,dc=dk`.
4. **Security settings** (same tab): login method *Fixed list*, and replace the example admin DN with `cn=admin,dc=nordtech,dc=dk`.
5. **Account types tab:** set the **LDAP suffix** for *Users* to `ou=people,dc=nordtech,dc=dk` and for *Groups* to `ou=groups,dc=nordtech,dc=dk`.
6. **Change the profile password** from the default `lam` to something else, so nobody can edit your profile. Save.
7. Back on the login page, log in with the **LDAP admin password** (the one from step 1.2).

> LAM suggests UID and GID numbers starting at about 1000 by default. **Do not accept them.** Type your own numbers (UIDs from 10000, GIDs from 5000) when you create each account, or change the ranges in the profile's module settings.

Create the groups

Go to the **Groups** tab and choose **New group**. For each group, fill in the Unix tab with the group name and the GID, then save:

Group	GID
`ldapadmins`	5000
`sales`	5001
`economy`	5002
`support`	5003

Create the users

Go to the **Users** tab and choose **New user**. For each user, fill in the *Personal* tab (first and last name) and the *Unix* tab (user name, UID, primary group, home directory, login shell), then set an **initial password** (use the same one for all users in this lab, for example `NordTech-2026!`) and save.

> **Where is the password field?** In LAM the password is not a normal text field. Click the **Set password** button, type the initial password twice, and save the user. **Do this for every user** (maria, jens, lars, and later sofie), and leave the **Force password change** box unticked (see below). A user without a password cannot log in. If you cannot find the button in your LAM version, create the user without a password and set it afterwards with `ldappasswd` (see step 1.6).

User	User name	UID	Primary group	Home	Shell
Maria Hansen	`maria`	10001	`sales`	`/home/maria`	`/bin/bash`
Jens Nielsen	`jens`	10002	`economy`	`/home/jens`	`/bin/bash`
Lars Madsen	`lars`	10003	`support`	`/home/lars`	`/bin/bash`

Make `lars` also a member of `ldapadmins` (edit the user and add the group under *additional groups* in the Unix tab, or edit the group and add him as a member).

Forced password change at first login (not used)

It is possible in principle to make users change the initial password at their first login. LAM has a **Force password change** box (it sets `shadowLastChange: 0` on the user), and SSSD can read the shadow attributes with `ldap_pwd_policy = shadow`. **We tested this and it did not work**: with that SSSD setting, logins failed. We therefore do not use it in this exercise. Set a password for every user with the **Set password** button as described above, and leave the box unticked.

> LAM's screens differ a little between versions. If you cannot find a field, use LAM's *Tree view* (under *Tools*) to see the raw entry and compare it with the LDIF below.

1.6 Alternative: create groups and users with LDIF files

If LAM does not work for you, or you want to see what LAM does behind the scenes, you can create the same content on the command line. Make **`groups.ldif`** and **`users.ldif`**:

`groups.ldif`: four groups.

```
dn: cn=ldapadmins,ou=groups,dc=nordtech,dc=dk
objectClass: posixGroup
cn: ldapadmins
gidNumber: 5000
memberUid: lars

dn: cn=sales,ou=groups,dc=nordtech,dc=dk
objectClass: posixGroup
cn: sales
gidNumber: 5001
memberUid: maria

dn: cn=economy,ou=groups,dc=nordtech,dc=dk
objectClass: posixGroup
cn: economy
gidNumber: 5002
memberUid: jens

dn: cn=support,ou=groups,dc=nordtech,dc=dk
objectClass: posixGroup
cn: support
gidNumber: 5003
memberUid: lars
```

`users.ldif`: three users.

```
dn: uid=maria,ou=people,dc=nordtech,dc=dk
objectClass: inetOrgPerson
objectClass: posixAccount
objectClass: shadowAccount
uid: maria
cn: Maria Hansen
givenName: Maria
sn: Hansen
uidNumber: 10001
gidNumber: 5001
homeDirectory: /home/maria
loginShell: /bin/bash

dn: uid=jens,ou=people,dc=nordtech,dc=dk
objectClass: inetOrgPerson
objectClass: posixAccount
objectClass: shadowAccount
uid: jens
cn: Jens Nielsen
givenName: Jens
sn: Nielsen
uidNumber: 10002
gidNumber: 5002
homeDirectory: /home/jens
loginShell: /bin/bash

dn: uid=lars,ou=people,dc=nordtech,dc=dk
objectClass: inetOrgPerson
objectClass: posixAccount
objectClass: shadowAccount
uid: lars
cn: Lars Madsen
givenName: Lars
sn: Madsen
uidNumber: 10003
gidNumber: 5003
homeDirectory: /home/lars
loginShell: /bin/bash
```

Import them with the admin password, **groups first**:

```
ldapadd -x -D "cn=admin,dc=nordtech,dc=dk" -W -f groups.ldif
ldapadd -x -D "cn=admin,dc=nordtech,dc=dk" -W -f users.ldif
```

Give each user a password:

```
ldappasswd -x -D "cn=admin,dc=nordtech,dc=dk" -W -S \
  "uid=maria,ou=people,dc=nordtech,dc=dk"
ldappasswd -x -D "cn=admin,dc=nordtech,dc=dk" -W -S \
  "uid=jens,ou=people,dc=nordtech,dc=dk"
ldappasswd -x -D "cn=admin,dc=nordtech,dc=dk" -W -S \
  "uid=lars,ou=people,dc=nordtech,dc=dk"
```

`-W` asks for the **admin** password. `-S` asks for the **new user** password. Use the same initial password for everyone. (Skip the `ldappasswd` commands if you set the passwords in LAM.)

Compare the two approaches: open one of your LAM-created users with *Tree view*, or with `ldapsearch`, and see that it contains the same attributes as the LDIF entry.

1.7 Verify from the server

```
ldapsearch -x -LLL -H ldap://ldap.nordtech.dk -b "dc=nordtech,dc=dk" \
  "(uid=maria)"
ldapwhoami -x -D "uid=maria,ou=people,dc=nordtech,dc=dk" -W
```

The first command should list Maria's entry. The second asks for Maria's password and should answer with her DN.

> **Checkpoint:** do not continue until both commands work.

Part 2: Desktop 1 as an LDAP client

All commands in Part 2 run on **Desktop 1**.

2.1 Make the server reachable

Add the server to `/etc/hosts`:

```
192.168.X.X   ldap.nordtech.dk   ldap
```

Test the connection:

```
ldapsearch -x -LLL -H ldap://ldap.nordtech.dk -b "dc=nordtech,dc=dk" \
  "(uid=maria)"
```

If this fails, fix it before going on. SSSD cannot work if this does not.

Note that you ask **anonymously** (there is no `-D`), and still get an answer. The default directory is open to everyone. In step 4.6 you can read how that door is closed.

2.2 Install SSSD

```
sudo apt update
sudo apt install sssd-ldap sssd-tools ldap-utils libpam-sss libnss-sss
```

2.3 Configure SSSD

Create `/etc/sssd/sssd.conf`:

```
sudo nano /etc/sssd/sssd.conf
```

```
[sssd]
config_file_version = 2
domains = nordtech.dk

[domain/nordtech.dk]
id_provider = ldap
auth_provider = ldap
ldap_uri = ldap://ldap.nordtech.dk
ldap_search_base = dc=nordtech,dc=dk
cache_credentials = True
ldap_auth_disable_tls_never_use_in_production = true
ldap_id_use_start_tls = false
```

About the two TLS lines. By default SSSD tries to use an encrypted connection (START_TLS), both when it looks users up (`ldap_id_use_start_tls`) and when it checks passwords. Your lab server has no certificate yet, so `sssd` logs `Could not start TLS encryption` and lookups fail. These two lines turn encryption off so the lab works for now. **Never use it in production**, because passwords would cross the network unencrypted. In Part 5 you will see exactly what that means (step 5.1), and in the optional steps 5.2 to 5.5 you fix it.

The file must be readable only by root, or SSSD refuses to start:

```
sudo chmod 600 /etc/sssd/sssd.conf
sudo chown root:root /etc/sssd/sssd.conf
sudo systemctl enable --now sssd
```

2.4 Create home directories automatically

```
sudo pam-auth-update --enable mkhomedir
```

2.5 Check name resolution

Open `/etc/nsswitch.conf` and check that `sss` appears on the `passwd` and `group` lines. If it does not, add it:

```
passwd:         files systemd sss
group:          files systemd sss
```

2.6 Test

```
getent passwd maria
id maria
su - maria
```

- `getent passwd maria` should print a line with UID **10001**. Compare with `grep maria /etc/passwd`. There is no match in the local file, yet the system knows the user.
- `su - maria` should ask for Maria's password and then log you in. It creates `/home/maria` on the first login. **Run it as an ordinary user, not as root**: root can `su` to anyone without a password, so the password prompt is skipped. Logging in on the console or with `ssh maria@desktop1` also works.
- Try `getent passwd` without a name. SSSD does not list all LDAP users by default. It looks users up one at a time.

Checkpoint: an LDAP user can log in on Desktop 1.

Part 3: Desktop 2

Repeat **Part 2 exactly** on **Desktop 2**.

When it works, run `getent passwd maria` on both desktops. The UID must be the same on both machines. This is the whole point of the next part.

Part 4: Managing things centrally

4.1 Add a user in one place

On the **server**, add a fourth user, `sofie` (Sofie Larsen, UID 10004, primary group `support`, home `/home/sofie`). Use LAM (**Users → New user**), or a new LDIF file with `uidNumber: 10004` and a password set with `ldappasswd`.

Now, on **both desktops**, run `getent passwd sofie` and log in as that user. You touched only the server.

> If a new user does not show up, SSSD may have cached an old answer. Run `sudo sss_cache -E` on the client (the command comes from the `sssd-tools` package installed in step 2.2).

4.2 Central offboarding

Imagine `jens` is a contractor whose contract has ended. In LAM, edit the user, open the Unix tab, and change the login shell to `/usr/sbin/nologin`. Or, on the **server**, write `disable_jens.ldif`:

```
dn: uid=jens,ou=people,dc=nordtech,dc=dk
changetype: modify
replace: loginShell
loginShell: /usr/sbin/nologin
```

```
ldapmodify -x -D "cn=admin,dc=nordtech,dc=dk" -W -f disable_jens.ldif
```

Clear the cache on both desktops (`sudo sss_cache -E`) and try `su - jens`. Compare with the work you did in lesson 5, where you had to touch every machine, and where a forgotten account could stay active.

4.3 Sudo rights through a central group

The idea. In lesson 3 you gave sudo rights to a *user* in a file on one machine. Now the group `ldapadmins` lives in the **directory**, and you only write one rule on each machine that says "everyone in this group may use sudo". Who is *in* the group is decided centrally. The rule itself is still a file on each desktop.

`lars` is a member of `ldapadmins`.

Step 1: check that the desktop can see the group. On **each desktop**:

```
getent group ldapadmins
id lars
```

`getent group ldapadmins` should list `lars` as a member, and `id lars` should include `ldapadmins` among his groups. If not, fix SSSD first (`sudo sss_cache -E`), because sudo can only use groups that the system can see.

Step 2: create the sudo rule. On **each desktop**, open a new file with `visudo`:

```
sudo visudo -f /etc/sudoers.d/90-ldapadmins
```

Part	Meaning
`sudo`	Run as root, because only root may edit sudo rules.
`visudo`	A special editor for sudo rules. It locks the file while you edit and **checks the syntax when you save**.
`-f /etc/sudoers.d/90-ldapadmins`	Edit this **file** instead of the main `/etc/sudoers`. If it does not exist, it is created.

Why a file in `/etc/sudoers.d/`? The main `/etc/sudoers` includes every file in that folder. A separate file keeps your rule apart from Ubuntu's defaults, so updates do not overwrite it, and you can remove it by deleting one file.

The name `90-ldapadmins` matters:

- Files are read in alphabetical order, and later rules win. The `90-` makes sure it is read after the defaults.
- Files whose names contain a `.` or end in `~` are **ignored**. Do not call it `90-ldapadmins.conf`.

Type this single line in the editor, then save and exit (in `nano`: `Ctrl+O`, `Enter`, `Ctrl+X`):

```
%ldapadmins ALL=(ALL:ALL) ALL
```

Part	Meaning
`%ldapadmins`	The `%` means a **group**, so everyone in the group `ldapadmins`. Without `%` it would be a single user.
`ALL=`	On all hosts.
`(ALL:ALL)`	They may run commands as any user and any group.
`ALL`	They may run any command.

The group comes from the LDAP directory through SSSD. The rule is stored locally on each desktop, so you must create the file on **both** desktops. The directory decides who is in the group, but the rule that gives the group power lives on each machine.

Step 3: test. As an ordinary user on a desktop (not as root):

```
su - lars
sudo whoami      # should print: root
exit
```

Then try the same as `maria` (`su - maria`, then `sudo whoami`). It should be refused, because she is not in `ldapadmins`.

Step 4: think about it. Who decides who is an administrator now: the sudoers file on the machine, or the directory? To add another administrator, which machine(s) do you have to touch? To change *what* administrators may do, which machine(s) do you have to touch?

4.4 File ownership and ACLs

Files refer to **numbers**, not names. On Desktop 1:

```
sudo mkdir /srv/shared
sudo touch /srv/shared/report.txt
sudo chown maria:sales /srv/shared/report.txt
sudo setfacl -m u:lars:r /srv/shared/report.txt
ls -ln /srv/shared/report.txt
getfacl /srv/shared/report.txt
```

`ls -ln` shows the raw UID and GID. Repeat the commands on Desktop 2. Then answer: what would go wrong if Desktop 2 had given `maria` a different UID? Why does a central directory make permissions and ACLs trustworthy across machines?

4.5 What is *not* centralized

Now test things we configured in lesson 5 and see how they behave with LDAP users.

Account lockout. Follow your faillock setup from lesson 5 on **both desktops**.

> **Important:** install SSSD and run `pam-auth-update` **first**, and configure faillock **afterwards**. `pam-auth-update` regenerates `/etc/pam.d/common-auth` and can wipe your manual edits.

With SSSD installed, `common-auth` contains a `pam_sss.so` line as well, so the faillock lines must go around **both** modules. A layout like this is expected (compare with your own file, since the exact `success=` numbers depend on it):

```
auth    required                        pam_faillock.so preauth silent
auth    [success=2 default=ignore]      pam_unix.so nullok
auth    [success=1 default=ignore]      pam_sss.so use_first_pass
auth    [default=die]                   pam_faillock.so authfail
auth    sufficient                      pam_faillock.so authsucc
auth    requisite                       pam_deny.so
auth    required                        pam_permit.so
```

Then:

1. On Desktop 1, log in as an LDAP user with three wrong passwords. Check with `sudo faillock --user maria`.
2. Is the account locked on Desktop 1?
3. Try to log in as the same user on Desktop 2. What happens?
4. Reset when you are done: `sudo faillock --user maria --reset`.

Password policy and MFA. Which of these live in the directory, and which are still files on each machine: `pwquality.conf`, `faillock.conf`, `/etc/pam.d/common-auth`, the `~/.google_authenticator` file?

Password policy: explained (trying it is optional).

The password rules from lesson 5 are in `/etc/security/pwquality.conf` on **each machine**, and `pam_pwquality` checks them locally when a user runs `passwd`, before anything is sent to the LDAP server. So the directory knows Maria, but each desktop decides on its own what a good password is. Two desktops can enforce different rules for the same LDAP user.

LAM also has password policy settings, but they only apply when an administrator sets a password **in the LAM web page**. A user who runs `passwd` on a desktop never goes through LAM. A policy only counts where it is enforced.

A central policy is possible: OpenLDAP has a password policy overlay (`ppolicy`). One policy, stored in the directory and enforced by the server, applies to every client, and failed logins are counted on the server, so a lockout applies on all machines (the opposite of what you saw with faillock). **We do not set this up in this exercise.**

Optional, if you want to see the local behavior yourself:

1. On **Desktop 2**, set a stricter rule in `/etc/security/pwquality.conf` (remove the `#` in front of `minlen` and set the value):

   ```
   minlen = 14
   ```

2. On **each desktop**, run `su - maria` as an ordinary user, and then `passwd`. When asked for the new password, try one of 10 characters.
3. Compare: Desktop 1 (with Ubuntu's default `minlen`) accepts it in the local check, while Desktop 2 refuses it with a message like `BAD PASSWORD: The password is shorter than 14 characters`.
4. Put `minlen` back to the old value on Desktop 2 when you are done.

> If the password is accepted by the local check, the change may still fail afterwards at the LDAP step, with an error such as *Authentication token manipulation error*. That is the missing TLS (the optional steps 5.2 to 5.5) and is not part of this test. Only the rejection message matters here.

Discuss: why was Maria treated differently on the two desktops, and which file decided it?

4.6 Least privilege for the directory itself (explained, not set up)

> **This part is only explained.** You do not set it up in this exercise. The aim is to understand how you would secure the LDAP server, and why.

In lesson 3 you gave administrators only the rights they need. The directory deserves the same treatment. Look at who uses it in your lab right now:

Who	Logs in as	What it can do today
You, and LAM	`cn=admin`	Everything
The desktops (SSSD)	Nobody (anonymous)	Read every user and group
Anyone on the network	Nobody (anonymous)	Read every user and group

There are two problems. The desktops only need to **look things up**, yet the same open door lets anyone on the network list all your staff, their UIDs, home directories and groups, which is a perfect starting point for an attacker. And there is no separation between "look up Maria" and "change Maria".

To secure the LDAP server you would use one account per job.

Account	Used by	What it may do
`cn=admin`	Only in an emergency ("break glass")	Everything. Strong password, used rarely, never in scripts.
A limited administrator account	The people who manage users, for example through LAM	Create and edit users and groups. **No** access to the ACLs and the server configuration.
`cn=sssd` (a read-only service account)	The desktops	Look up users and groups. Never read the password hashes, never write.
An ordinary user	Themselves	Change their own password and read their own entry.
Anonymous	Nobody	Nothing, except trying to log in.

This is the same idea as `sudo` in lesson 3: nobody works as root all day, and a service gets exactly the access it needs and nothing more.

How it is done in OpenLDAP: ACLs. The directory's access rules are called **ACLs**. If you are curious, you can look at the ones on your own server:

```
sudo ldapsearch -Q -LLL -Y EXTERNAL -H ldapi:/// -b cn=config \
  "(olcAccess=*)" dn olcAccess
```

The output has three blocks. The first two (`frontend` and `config`) are the server's own settings and only allow the operating system's root account to manage them. The block that matters is the one with `dn: olcDatabase={1}mdb,cn=config`: that is the database with your users. It has three rules, like this:

```
olcAccess: {0}to attrs=userPassword by self write by anonymous auth
  by * none
olcAccess: {1}to attrs=shadowLastChange by self write by * read
olcAccess: {2}to * by * read
```

Read each one as "**to** what, **by** whom, which right". The rules are checked from the top, and the first rule that matches the data decides:

Rule	Meaning
`{0}`	The password attribute: the user may change **their own** (`self write`), anonymous users may only **try to log in** (`auth`), and everybody else gets **nothing**. This is why nobody can read the password hashes.
`{1}`	The `shadowLastChange` attribute: the user may write their own, and anyone may read it.
`{2}`	**Everything else: everybody may read it.** This is the open door.

The administrator `cn=admin` is not in the list, because the server's root account is always allowed to do everything. That is exactly why it should be used rarely.

To close the door, you replace the last rule with one like this: the read-only account may read, a user may read their own entry, and everybody else gets nothing.

```
olcAccess: {2}to * by dn.exact="cn=sssd,ou=services,dc=nordtech,dc=dk"
  read by self read by * none
```

What the desktops need. SSSD must then identify itself instead of asking anonymously. In `sssd.conf` it would get three extra lines:

```
ldap_default_bind_dn = cn=sssd,ou=services,dc=nordtech,dc=dk
ldap_default_authtok_type = password
ldap_default_authtok = THE-PASSWORD-OF-THE-ACCOUNT
```

Why is the password written in plain text here, and not as a hash? On the server the password is stored as a hash, and the server only compares hashes. But SSSD has to **send** the real password when it logs in, and the server hashes what it receives. If you wrote the hash in `sssd.conf`, SSSD would send the hash as the password, the server would hash it a second time, and the login would fail. Every client that proves its identity with a password must know that password. That is why the file must be readable only by root (mode 600), why the account is read-only (a stolen file gives an attacker almost nothing), and why the connection should be encrypted with TLS (Part 5). SSSD can "obfuscate" the password with `sss_obfuscate`, but that only hides it from a quick glance and is easy to reverse, so it is not real protection.

What you would see with this in place.

- An anonymous search for users returns nothing.
- The read-only account sees the users, but never the `userPassword` attribute.
- If the read-only account tries to change an entry, the server answers `Insufficient access (50)`. The account that every desktop knows the password of cannot damage the directory.
- Maria can read her own entry, but not Jens'. The directory does not even admit that his entry exists.

Think about it.

- Why should `sssd.conf` use a read-only account and not `cn=admin`? What can an attacker do who steals a desktop and reads that file, compared with what they could do if the file held the admin password?
- LAM still logs in as `cn=admin` in this exercise, so everything you do in the web page is done with full control. Which account would you give the people who manage users instead?
- Why is the administrator account not simply deleted, if it is so powerful?

Part 5: Encrypt the connection with TLS

So far the lab has been **unencrypted**. Step 5.1 shows what that means and is part of the exercise. Steps 5.2 to 5.5 (fixing it with TLS) are optional.

5.1 See the problem

You will capture the network traffic on a **desktop** into a file, and open the file in Wireshark.

On **Desktop 2**, install the tools. When the installer asks whether non-root users may capture packets, answer **No**.

```
sudo apt install tcpdump wireshark
```

Open **two terminals** on Desktop 2. In the first, start the capture:

```
sudo tcpdump -i any -s0 -w ~/ldap-plain.pcap 'tcp port 389'
```

In the second, make SSSD talk to the server, and log in as a user:

```
sudo systemctl restart sssd
sudo sss_cache -E
su - sofie
```

Type the password, then `exit`, and stop the capture in the first terminal with `Ctrl+C`. Make the file readable for your own user:

```
sudo chown $USER ~/ldap-plain.pcap
```

Open it in Wireshark:

```
wireshark ~/ldap-plain.pcap
```

Type `ldap` in the display filter at the top. Find a **bindRequest**, open it, and expand *Lightweight Directory Access Protocol → protocolOp → bindRequest → authentication*. Can you read the password you typed? You can also right-click a packet and choose *Follow → TCP Stream*. This is what anyone on the network path can see.

5.2 Create a certificate authority and a server certificate

> **Steps 5.2 to 5.5 are optional.** Do them if you have time, or if you want to see the fix with your own eyes.

> **Tip: log in to the server with `ssh`.** The commands in 5.2 are long and the server console does not let you copy and paste. From a desktop (or your own computer), connect with `ssh YOUR_USER@ldap.nordtech.dk` (use the user name you created when installing the server). Then you can paste the whole command block in one go, which also avoids typing mistakes. If `ssh` says the connection is refused, install the SSH server on the **server** first: `sudo apt install openssh-server`.

On the **server**, in an empty working directory:

```
mkdir ~/tls && cd ~/tls

# 1. A small certificate authority (CA) for the lab
openssl genrsa -out ca.key 4096
openssl req -x509 -new -nodes -key ca.key -sha256 -days 365 \
  -subj "/CN=NordTech Lab CA" -out ca.crt

# 2. A key and certificate request for the LDAP server
openssl genrsa -out ldap.key 4096
openssl req -new -key ldap.key -subj "/CN=ldap.nordtech.dk" -out ldap.csr

# 3. Sign it with the CA. The name MUST match ldap.nordtech.dk
cat > san.cnf <<'EOF'
subjectAltName = DNS:ldap.nordtech.dk
extendedKeyUsage = serverAuth
EOF
openssl x509 -req -in ldap.csr -CA ca.crt -CAkey ca.key -CAcreateserial \
  -out ldap.crt -days 365 -sha256 -extfile san.cnf
```

Install the files where slapd can read them:

```
sudo cp ca.crt /etc/ssl/certs/nordtech-ca.crt
sudo cp ldap.crt /etc/ssl/certs/ldap.crt
sudo cp ldap.key /etc/ssl/private/ldap.key
sudo chgrp ssl-cert /etc/ssl/private/ldap.key
sudo chmod 640 /etc/ssl/private/ldap.key
sudo usermod -aG ssl-cert openldap
sudo systemctl restart slapd
```

The restart is needed because a running program only picks up a new group membership when it starts. Without it, slapd cannot read the key file in the next step.

5.3 Tell slapd to use them

Create **`tls.ldif`**:

```
dn: cn=config
changetype: modify
replace: olcTLSCACertificateFile
olcTLSCACertificateFile: /etc/ssl/certs/nordtech-ca.crt
-
replace: olcTLSCertificateFile
olcTLSCertificateFile: /etc/ssl/certs/ldap.crt
-
replace: olcTLSCertificateKeyFile
olcTLSCertificateKeyFile: /etc/ssl/private/ldap.key
```

```
sudo ldapmodify -Y EXTERNAL -H ldapi:/// -f tls.ldif
sudo systemctl restart slapd
```

Test that TLS works from the server, with `-ZZ` (START_TLS, and fail if it does not work):

```
LDAPTLS_CACERT=/etc/ssl/certs/nordtech-ca.crt \
  ldapwhoami -x -ZZ -H ldap://ldap.nordtech.dk -D \
    "uid=maria,ou=people,dc=nordtech,dc=dk" -W
```

> **Checkpoint:** do not continue until this command works.

5.4 Let the clients trust the CA and use TLS

Copy `ca.crt` from the server to **both desktops** (for example with `scp`) and install it:

```
sudo cp ca.crt /etc/ssl/certs/nordtech-ca.crt
```

On **both desktops**, edit `/etc/sssd/sssd.conf`. **Remove** the lines `ldap_auth_disable_tls_never_use_in_production = true` and `ldap_id_use_start_tls = false`, and add:

```
ldap_id_use_start_tls = true
ldap_tls_cacert = /etc/ssl/certs/nordtech-ca.crt
ldap_tls_reqcert = demand
```

Restart and clear the cache:

```
sudo systemctl restart sssd
sudo sss_cache -E
su - maria
```

5.5 Prove that it works

Capture again on Desktop 2, now into a new file:

```
sudo tcpdump -i any -s0 -w ~/ldap-tls.pcap 'tcp port 389'
```

In the second terminal, log in again:

```
sudo systemctl restart sssd
sudo sss_cache -E
su - sofie
```

Stop the capture, run `sudo chown $USER ~/ldap-tls.pcap`, and open it with `wireshark ~/ldap-tls.pcap`. Try the filters `ldap` and `tls`. You should only see the START_TLS request (an *extendedReq*) followed by *TLS* packets with unreadable contents. Compare it with `ldap-plain.pcap` from step 5.1: where is the password now?

Finally, answer: why does `ldap_tls_reqcert = demand` matter? What could an attacker do if the client accepted *any* certificate?

Part 6: When the server fails, and protecting the directory

> **This part is optional.** Do it if you have time.

6.1 The LDAP server is down

First make sure `maria` has logged in on Desktop 1 at least once (you did this in Part 2). Then, on the **server**:

```
sudo systemctl stop slapd
```

On **Desktop 1**, try:

```
getent passwd maria
su - maria
su - sofie     # sofie has never logged in on this desktop
```

Which of them works, and why? What does `cache_credentials = True` in `sssd.conf` do? Then bring the server back:

```
sudo systemctl start slapd
```

Think about the 300-machine company: what would you do so that one dead server does not stop everyone from logging in? (Hint: replication.)

6.2 Who can read the passwords?

Run an anonymous search from the server, asking for the password attribute:

```
ldapsearch -x -LLL -H ldap://ldap.nordtech.dk -b "dc=nordtech,dc=dk" \
  "(uid=maria)" userPassword
```

Then run the same search as the administrator:

```
ldapsearch -x -LLL -D "cn=admin,dc=nordtech,dc=dk" -W -H \
  ldap://ldap.nordtech.dk -b "dc=nordtech,dc=dk" "(uid=maria)" \
  userPassword
```

What is the difference? The directory's **ACL** (access control list, see step 4.6) decides who may read which attribute. Only the administrator sees the stored value. Note that it starts with `{SSHA}` or similar: it is a salted hash, just like the ones you saw in `/etc/shadow` in lesson 5.

6.3 Logging

Turn on logging of LDAP operations. On the **server**, write `loglevel.ldif`:

```
dn: cn=config
changetype: modify
replace: olcLogLevel
olcLogLevel: stats
```

```
sudo ldapmodify -Y EXTERNAL -H ldapi:/// -f loglevel.ldif
```

Log in as `maria` on a desktop and watch the server:

```
sudo journalctl -u slapd -f
```

You should see connections, binds and searches. Who connected, from which IP, and what did they search for?

6.4 Backup

```
sudo slapcat -b "dc=nordtech,dc=dk" -l ~/nordtech-backup.ldif
head -30 ~/nordtech-backup.ldif
```

This is a plain-text export of the whole directory. Why must this file be protected as carefully as the server itself? When would you restore from it?

Part 7: Questions to think about

For everyone (Parts 1 to 4, and step 5.1)

1. Why do we use a central directory such as LDAP? Give two concrete problems it solves, and relate them to the Maria and Jens stories.
2. List the things you managed centrally in this exercise, and the things that remained local on each machine.
3. For one of the things that remained local, describe what a company with 300 machines would need to do to keep it consistent.
4. Explain what a **DN** is, and write the DN of one of your users and one of your groups.
5. Why did we use UIDs from 10000 and GIDs from 5000?
6. In the capture from 5.1, what could you read? Why must a real directory never run without TLS?
7. Name three things you would do to protect the LDAP server itself, and say why each one matters.
8. Which accounts should a directory have, and what may each one do (step 4.6)? Why should the desktops use a read-only account and not `cn=admin`? How is this the same idea as `sudo` in lesson 3?

If you did the optional TLS steps (5.2 to 5.5)

9. How did the capture in 5.5 differ from the one in 5.1? What does the certificate add on top of the encryption?

If you did the optional Part 6

10. What happened in 6.1 when the server was down? Which users could still log in, and what would you change so that logins survive a server failure?
11. What was the difference between the anonymous search and the administrator search in 6.2? Why must a backup file from 6.4 be protected as carefully as the server itself?

Troubleshooting

Problem	Try this
`ldapsearch` cannot connect	Check `/etc/hosts`, `ping ldap.nordtech.dk`, and that `slapd` is running: `systemctl status slapd`
`su - maria` says *Authentication failure*, and the log has `pam_sss ... received for user maria: 4 (System error)` and *Misconfiguration found for the 'pam' responder*	Remove the line `services = nss, pam` from `/etc/sssd/sssd.conf`, run `sudo systemctl reset-failed`, and restart `sssd`. Check with `systemctl is-active sssd-pam.socket`
`sss_cache: command not found`	Install the tools: `sudo apt install sssd-tools`
`getent passwd maria` shows the wrong home directory (for example `/home/jdoe`)	The `homeDirectory` attribute in the directory is wrong. Fix it in LAM (*Unix* tab) or with `ldapmodify`, then run `sudo sss_cache -E`. Do this **before** the user's first login, or the wrong home directory is created
`getent passwd maria` prints nothing	If `journalctl -u sssd` says `Could not start TLS encryption`, make sure `ldap_id_use_start_tls = false` is in the domain section (before the optional TLS steps 5.2 to 5.5). Check `systemctl status sssd` and `sudo journalctl -u sssd -b`. Check `sss` is in `/etc/nsswitch.conf`. Run `sudo sss_cache -E` and restart `sssd`
SSSD will not start	`/etc/sssd/sssd.conf` must be mode `600` and owned by root

Problem	Try this
User exists but the password is rejected	Re-set it with `ldappasswd`. Before the optional TLS steps (5.2 to 5.5), check that `ldap_auth_disable_tls_never_use_in_production = true` is in the config and restart `sssd`. After the TLS steps (5.2 to 5.5), check the TLS settings instead
Error `ldap_start_tls: Can't contact LDAP server`	The client is trying encryption the server does not offer, or does not trust. Before the optional TLS steps (5.2 to 5.5), use the lab setting. After the TLS steps (5.2 to 5.5), check that the CA file is on the client and that the server certificate is installed
`ldapadd` says "No such object"	Create the entries in the right order: `base.ldif` first, then groups and users
`ldapadd` says "Already exists"	The entry is already there. Check with `ldapsearch`, or delete it with `ldapdelete` and retry
Home directory is not created	Run `sudo pam-auth-update --enable mkhomedir` again and log in again
`http://SERVER_IP/lam` does not load	Check `systemctl status apache2`, that the server IP is correct, and the `Require ip` lines in the LAM Apache config
LAM login fails with the admin password	In the server profile, check that the admin DN is `cn=admin,dc=nordtech,dc=dk` and the tree suffix is correct
LAM says it cannot reach the LDAP server	Check the server address in the profile (`ldap://localhost:389` when LAM runs on the LDAP server) and that `slapd` is running
LAM shows an error about the `ou=people` or `ou=groups` suffix	Create the two OUs with `base.ldif` (step 1.4) and check the suffixes in the *Account types* tab
Locked yourself out after editing PAM files	Restore the snapshot, or the backup files you made before editing
slapd will not start after the TLS change	Check `sudo journalctl -u slapd -b`. The `openldap` user must be able to read `/etc/ssl/private/ldap.key` (group `ssl-cert`, mode `640`). If you changed the group, restart `slapd`
`ldapmodify` with `tls.ldif` fails with *Other (e.g., implementation specific) error (80)*	slapd could not load the certificate or key. Read the reason with `sudo journalctl -u slapd -n 20 --no-pager`. Usual causes: the `openldap` user cannot read `/etc/ssl/private/ldap.key` (add it to `ssl-cert` and run `sudo systemctl restart slapd`), a wrong path in `tls.ldif`, or the key does not match the certificate (compare with `openssl x509 -noout -modulus -in ldap.crt | md5sum` and `openssl rsa -noout -modulus -in ldap.key | md5sum`)
TLS test fails with a certificate or name error	The server name you connect to must match the certificate (`ldap.nordtech.dk`) and the `/etc/hosts` entry. Recreate the certificate with the right name in `san.cnf`
`ldapmodify` on `cn=config` is refused	Use `sudo` with `-Y EXTERNAL -H ldapi:///`, not `-x` and the admin password